"Develop a passion for learning."

MODULE 13

SOAP API Design with Java

Design, build, and consume enterprise-grade SOAP web services using Java and industry best practices.


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT

MODULE 13 OVERVIEW

Many enterprise organizations still rely on SOAP for contract-first design, strong security, and reliable messaging.

- SOAP (Simple Object Access Protocol) is a standards-based protocol for exchanging structured XML information — you'll design secure, contract-first SOAP service contracts in Java.

DURATION & LAB

4 Hours Theory

SOAP, XML, WSDL, XSD, security.

2 Hours Lab

SOAP API Design.

Scenario

Northstar CRM publishes a SOAP contract for a partner.

SOAP USES
XML messages, WSDL contracts, WS-* standards

BEST FOR
Systems that need contract-first design, strong security, and reliability

THE PAYOFF
Strong security, reliability, contract-first design

KEY TAKEAWAY SOAP isn't about being old — it's about being dependable. Right technology, right problem, right environment.

WHY SOAP STILL EXISTS

Despite the rise of REST, SOAP remains critical in enterprise and regulated environments.

Enterprise & Legacy

Mission-critical systems built on SOAP (mainframes, ESBs).

Strong Standards

Mature W3C/OASIS specs ensure interoperability.

Built-In Security

WS-Security: encryption, signatures, tokens.

Reliable & Transactional

WS-ReliableMessaging, WS-AtomicTransaction.

Contract-First

WSDL defines the contract before implementation.

Regulated Industries

Banking, insurance, healthcare, government.

REAL-WORLD EXAMPLES

- Legacy integration: stable SOAP endpoints let modern services talk to mainframes and ESBs without rewriting them.
- Compliance & auditability: WS-Security and WS-ReliableMessaging provide the audit trails and guaranteed delivery regulated processes require.

KEY TAKEAWAY SOAP isn't about being old — it's about being dependable when security, reliability, and compliance matter most.

Why SOAP Still Exists

SOAP VS REST

Two architectural styles for building APIs — they differ in philosophy, protocols, message formats, and use cases.

ASPECT	SOAP	REST
`Nature & Protocol`	Protocol-based, with strict rules. Typically HTTP, but also SMTP, JMS, TCP	An architectural style. REST APIs are most commonly implemented over HTTP/HTTPS.
`Message Format`	XML only	JSON, XML, YAML, etc. REST typically layers on HTTP caching (ETags, Cache-Control).
`Contract`	SOAP commonly uses formal WSDL contracts and often follows a contract-first approach.	REST APIs may be code-first or contract-first using OpenAPI.
`Security`	Built-in WS-Security; supports enterprise WS-* standards	HTTPS, OAuth 2.0, JWT, API Keys
`Performance`	SOAP XML messages are generally more verbose.	REST/JSON often produces smaller payloads, but actual performance should be measured.
`Java Example`	JAX-WS (Jakarta XML Web Services)	JAX-RS (Jakarta RESTful WS), Spring Web

CHOOSE BASED ON CONTRACT NEEDS, SECURITY MODEL, PAYLOAD SIZE, AND TEAM/TOOLING FIT — NOT PROTOCOL ALONE

KEY TAKEAWAY SOAP and REST fit different needs — evaluate contract requirements, security model, and payload characteristics for your use case, and measure performance rather than assuming it.

SOAP vs REST

Aspect	SOAP	REST
Approach	**Protocol-based** Strict standards and rules	**Architecture style** Guiding principles
Protocol	Uses **SOAP** protocol over HTTP, SMTP, etc.	Uses **HTTP** protocol (GET, POST, PUT, DELETE)
Data Format	**XML** only (Uses XSD for schema)	**JSON, XML, YAML, etc.** (Flexible)
Security	**WS-Security (built-in)** Supports enterprise security standards	**Relies on HTTP security** (OAuth, JWT, HTTPS, etc.)
Standards	**Heavy standards** (WSDL, XSD, SOAP)	**Lightweight** Leverages HTTP and URIs
Complexity	**Complex** More configuration and setup	**Simple** Easy to develop and consume
Performance	**Slower** Large XML payloads	**Faster** Lightweight payloads
Use Cases	**Enterprise systems** Banking, Insurance, Healthcare, Legacy integrations	**Web and mobile apps** Public APIs, Microservices, Cloud applications
Tooling	**Limited and heavier tools** (e.g., SOAP UI)	**Wide range of lightweight tools** (e.g., Postman, Insomnia)
Scalability	**Less scalable** Stateful operations	**Highly scalable** Stateless operations

SOAP ARCHITECTURE OVERVIEW

SOAP is a standard protocol for exchanging structured XML information in web services.

1 **Client Application** → 2 **SOAP Envelope** → 3 **Transport (HTTP/HTTPS)** → 4 **SOAP Service** → 5 **Response Returned**

1. **Client Application**

   Builds a SOAP request.

2. **SOAP Envelope**

   Header + Body wrap the message.

3. **Transport (HTTP/HTTPS)**

   Message sent to the service.

4. **SOAP Service**

   Processes the request.

5. **Response Returned**

   Client receives and parses it.

KEY COMPONENTS

WSDL

Defines the service contract.

UDDI (Historical)

Historical service-registry standard; rarely used in modern projects.

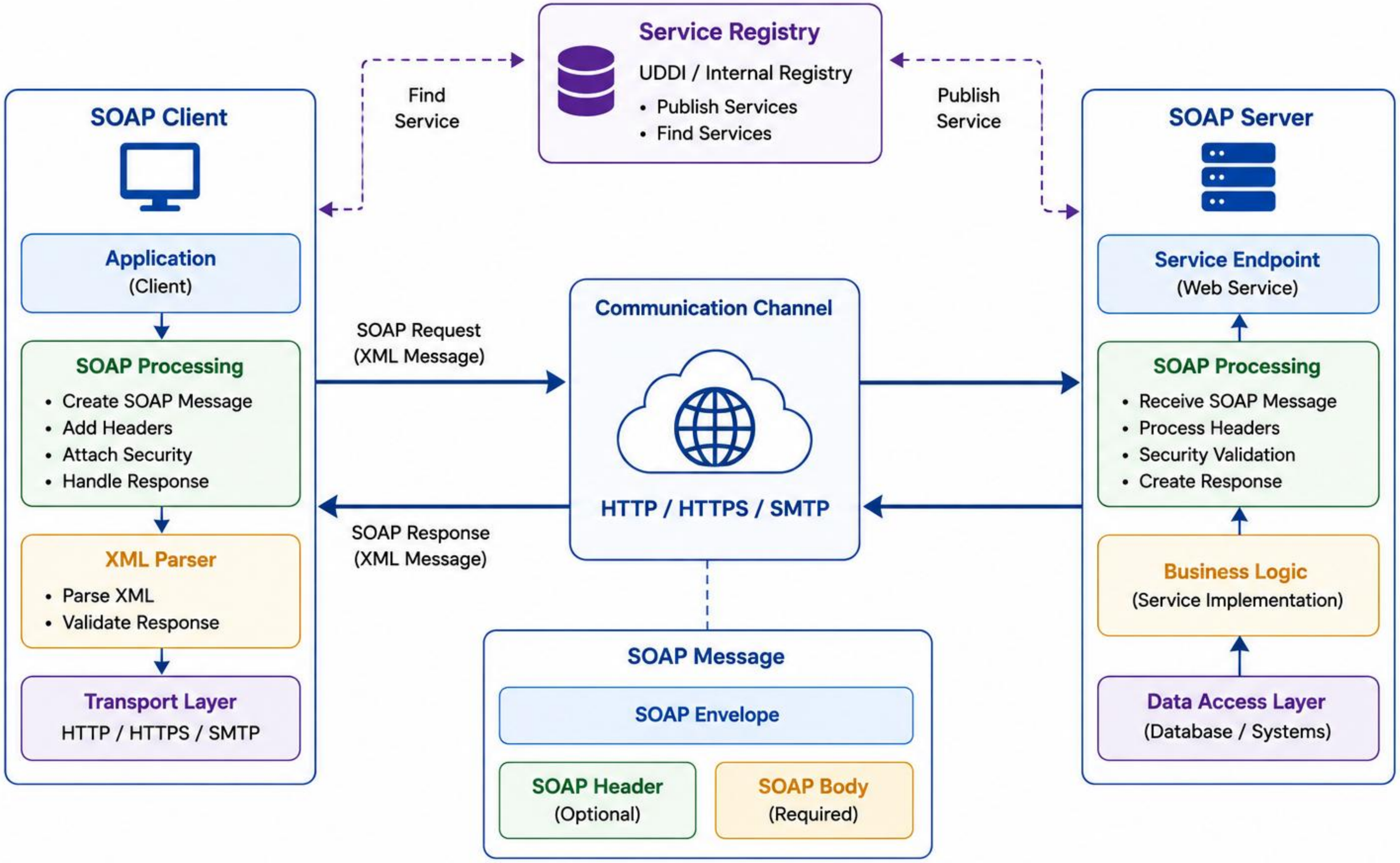
WS-*

Security, reliability, transactions.

WS- EXAMPLES: WS-Security (auth & encryption) • WS-ReliableMessaging (guaranteed delivery) • WS-AtomicTransaction (distributed transactions) • WS-Policy (policy assertions).*

KEY TAKEAWAY SOAP provides a robust, standards-based foundation for secure, reliable, interoperable web services.

SOAP Architecture



XML FUNDAMENTALS

XML is platform-independent, self-descriptive, and extensible — the foundation of SOAP, WSDL, and JAXB.

XML EXAMPLE

```xml
<?xml version="1.0" encoding="UTF-8"?>
<employee>
  <id>101</id>
  <name>John Doe</name>
  <role>Developer</role>
</employee>
```

XML BASIC RULES

- XML is case-sensitive; every element must have a closing tag.
- Use namespaces (xmlns) for namespace qualification and to avoid collisions.
- Declare character encoding (e.g., UTF-8) and escape special characters.
- Validate against an XSD schema; choose elements vs. attributes deliberately.

COMMON USE CASES

- **Data Exchange**
- **Namespace Qualification**
- **Web Services (SOAP)**
- **Schema Validation (XSD)**

Escape special characters (& < > " ') and use namespaces (xmlns) to avoid name collisions when combining XML from multiple sources.

KEY TAKEAWAY XML provides a flexible, standard way to represent structured data — the foundation for SOAP, WSDL, and JAXB.

XML Fundamentals

What is XML?

XML (eXtensible Markup Language) is a text-based markup language designed to store and transport data.

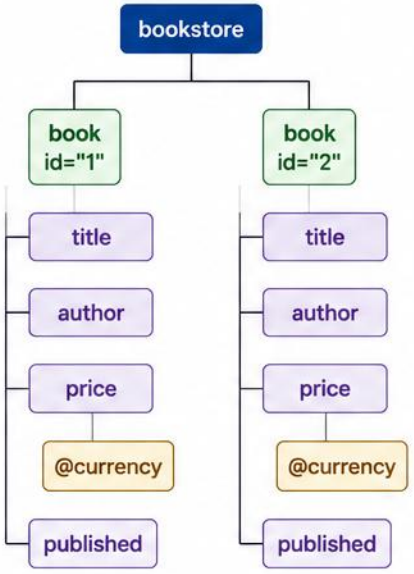
Extensible
Users define their own tags.

Portable
Platform and language independent.

Hierarchical
Organizes data in a tree structure.

XML Structure Example

```xml
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
 <book id="1">
    <title>Java Basics</title>
    <author>John Doe</author>
    <price currency="USD">29.99</price>
    <published>true</published>
 </book>
 <book id="2">
    <title>XML Guide</title>
    <author>Jane Smith</author>
    <price currency="USD">39.99</price>
    <published>false</published>
 </book>
</bookstore>
```



Root Element · Parent Element · Child Element · Attribute

Key XML Concepts

Elements
Defined by start and end tags.
`<tag>value</tag>`

Attributes
Provide additional information about elements.
`<tag attr="value">`

Text Content
The actual data inside elements.
`<tag>data</tag>`

Comments
Used to add notes (not displayed).
`<!-- comment -->`

Well-Formed XML Rules

- XML must have a single root element.
- All elements must have proper opening and closing tags.
- Tags are case-sensitive.
- All tags must be properly nested.
- Attribute values must be enclosed in quotes.
- Special characters must be escaped (<, >, &, ", ').

SOAP MESSAGE STRUCTURE

A SOAP message is an XML document that follows a specific structure — used to exchange information between client and service.

Envelope

- Root element that identifies the message as SOAP.

Header (Optional)

- Metadata: authentication, transactions, routing.

Body (Required)

- Contains the actual request or response data.

Fault (Inside Body)

- Nested inside Body when processing fails — not a sibling of Header/Body.

STRUCTURE RULES

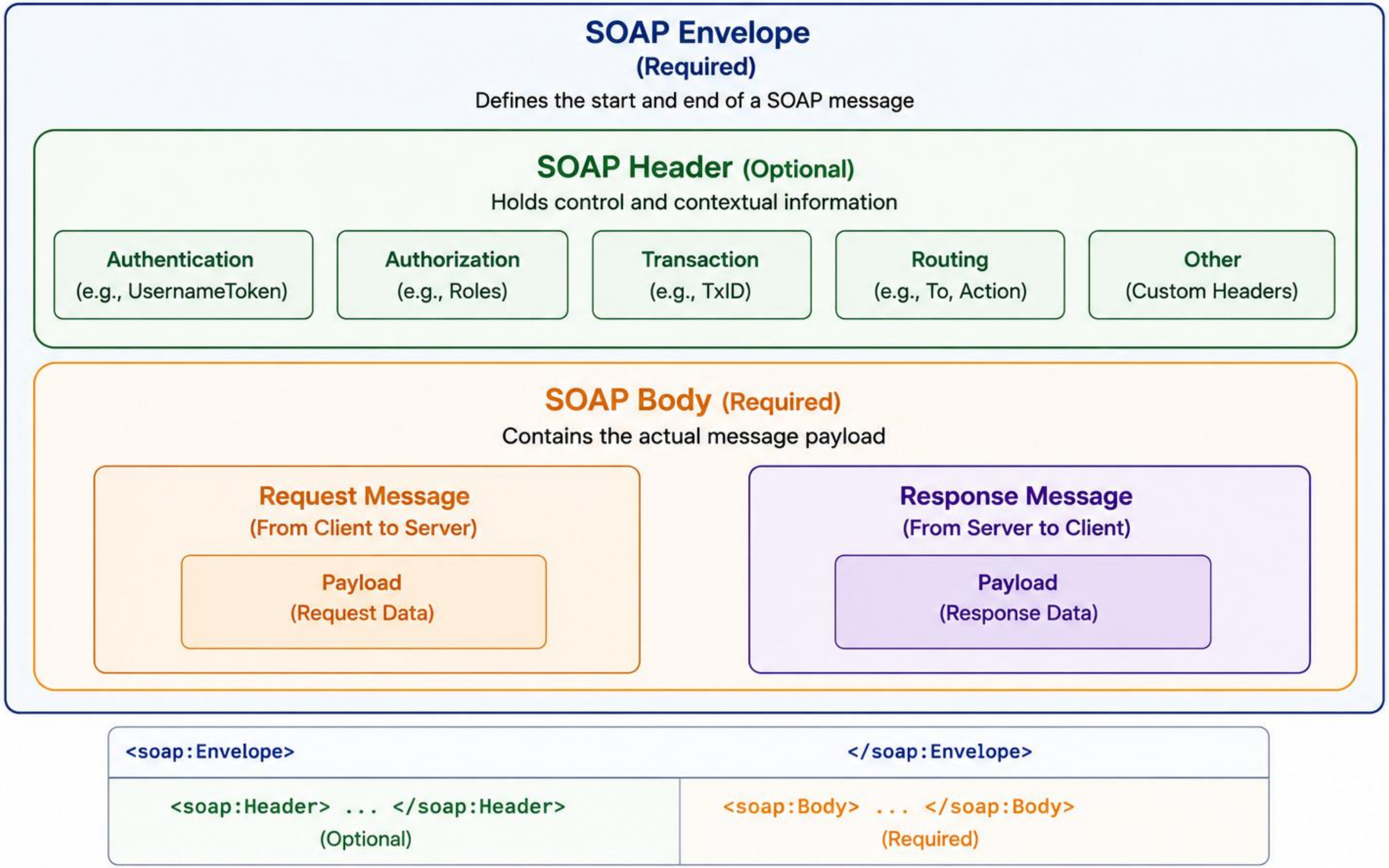
- The message must have a single <soap:Envelope> as the root.
- <soap:Header> is optional and comes before <soap:Body>.
- <soap:Body> is required; a <soap:Fault>, when present, is nested inside Body — not a sibling of Header/Body.

SOAP 1.1 and SOAP 1.2 differ in a few concrete ways:

Aspect	SOAP 1.1	SOAP 1.2
Envelope namespace	schemas.xmlsoap.org/...	www.w3.org/2003/05/...
HTTP content type	text/xml	application/soap+xml
Action	Usually SOAPAction header	Can be content-type parameter
Fault model	faultcode, faultstring, detail	Code, Reason, Detail

KEY TAKEAWAY SOAP messages are XML-based and strictly structured — validate against WSDL schemas and handle faults gracefully.

SOAP Message Structure



SOAP ENVELOPE

The root of every SOAP message — it defines namespaces and wraps Header, Body, and optional Fault.

MINIMAL SOAP ENVELOPE

```
<soap:Envelope xmlns:soap="...envelope/">
  <soap:Header>...</soap:Header>
  <soap:Body>
    <GetDataRequest>...</GetDataRequest>
  </soap:Body>
</soap:Envelope>
```

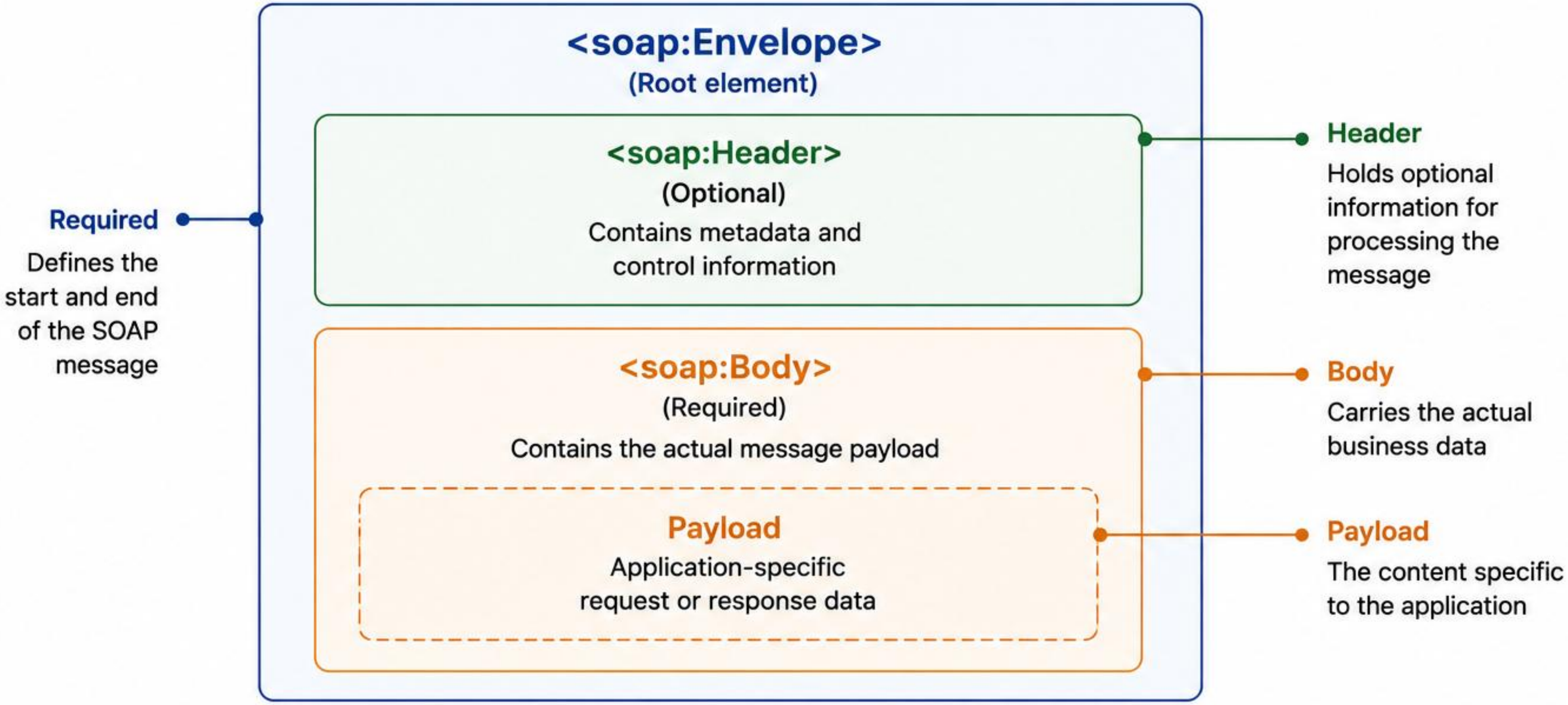
KEY RESPONSIBILITIES

- Defines namespace — declares the SOAP namespace.
- Encapsulates the message — wraps Header, Body, Fault.
- Provides structure — ensures a well-formed message.
- Enables interoperability across compliant systems.

Common namespace prefixes: soap (SOAP 1.1 envelope), soap12 (SOAP 1.2 envelope), xsi (XML Schema instance), xsd (XML Schema).

KEY TAKEAWAY The Envelope is mandatory and must be the first and last element — without it, the message is not valid SOAP.

SOAP Envelope




```xml
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">      <!-- SOAP Envelope Start -->
    <soap:Header>
        <!-- Optional header elements go here -->
    </soap:Header>                                                          <!-- Optional -->
    <soap:Body>
        <!-- Actual message payload goes here -->                           <!-- Required -->
    </soap:Body>
</soap:Envelope>                                                            <!-- SOAP Envelope End -->
```

HEADER, BODY, AND FAULT

Header carries optional metadata, Body carries the required payload, and Fault — nested inside Body — reports errors.

HEADER — SECURITY EXAMPLE

```
<soap:Header>
  <tns:AuthToken soap:mustUnderstand="1">
    <tns:Token>ABC123TOKEN</tns:Token>
  </tns:AuthToken>
</soap:Header>
```

BODY — REQUEST EXAMPLE

```
<soap:Body>
  <tns:CreateOrderRequest>
    <tns:CustomerId>1001</tns:CustomerId>
  </tns:CreateOrderRequest>
</soap:Body>
```

FAULT (INSIDE BODY) — SOAP 1.1 EXAMPLE

```
<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Invalid customer ID</faultstring>
  <detail><tns:ValidationFault><tns:code>CUSTOMER_ID_REQUIRED</tns:code>
  </tns:ValidationFault></detail>
</soap:Fault>
```

HEADER — USE CASES

- WS-Security tokens, routing, transactions, and QoS metadata.
- Custom-header example above is simplified — not a complete WS-Security implementation.
- Conceptual: <wsse:Security><wsse:UsernameToken>…</wsse:UsernameToken></wsse:Security>

BODY — KEY POINTS

- Required element in every SOAP message.
- For WS-I document/literal wrapped services, Body normally contains one operation wrapper element — follow the WSDL.
- Defined by the service contract (WSDL); can contain complex data structures.

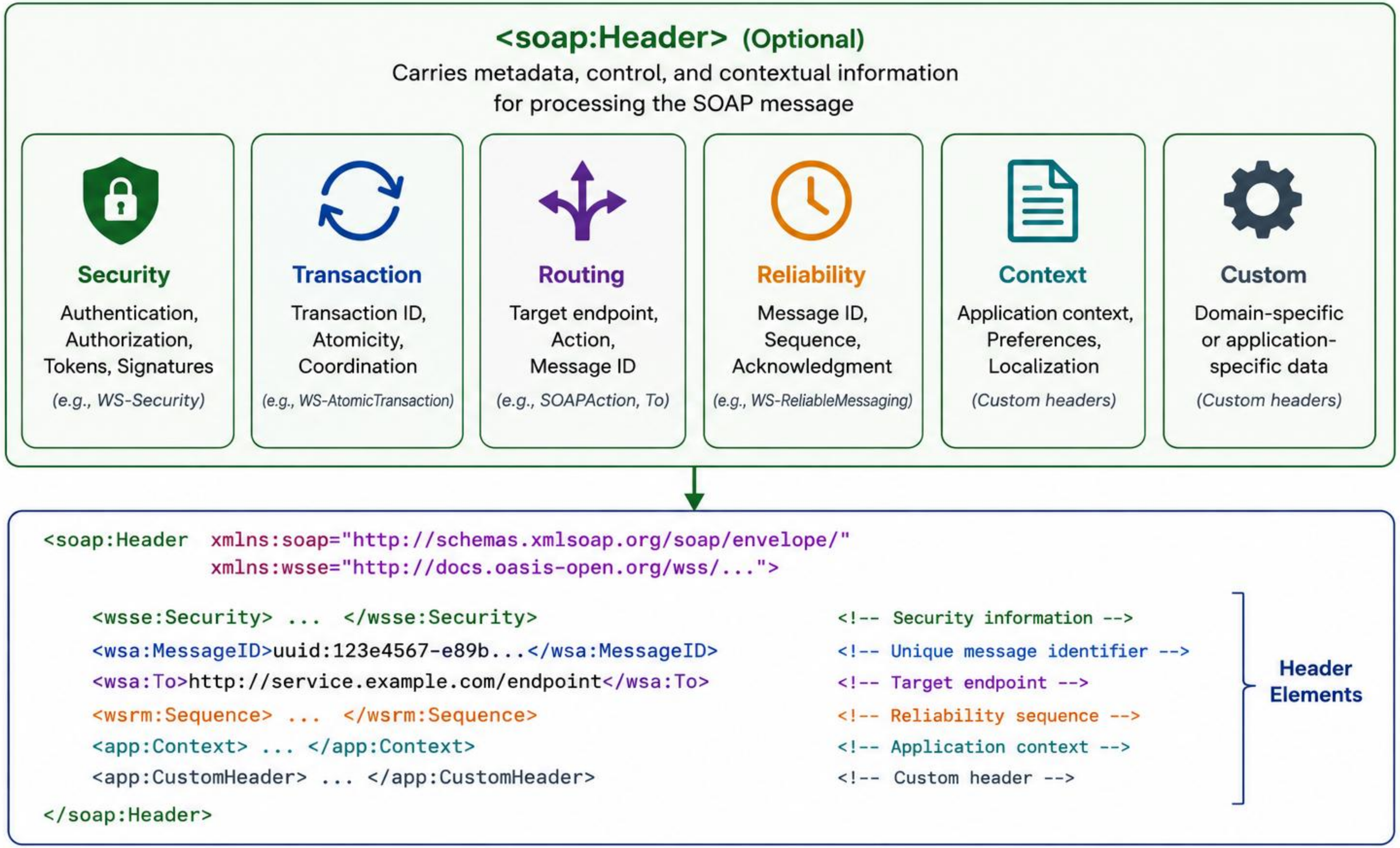
FAULT — KEY POINTS

- SOAP 1.1: faultcode (Client/Server), faultstring, and an optional detail element identify the error.
- SOAP 1.2 uses a different Fault structure: Code, Reason, and Detail instead of faultcode/faultstring/detail.

Key header attributes: mustUnderstand (forces processing or returns a Fault); soap:actor (SOAP 1.1) / soap:role (SOAP 1.2) target the intended recipient node. Body content types include simple elements, complex elements, collections, and custom XSD types.

KEY TAKEAWAY Header, Body, and Fault each play a distinct role — Fault always lives inside Body, never as a sibling of Header.

SOAP Header


<soap:Header> (Optional)
Carries metadata, control, and contextual information
for processing the SOAP message
Security
Authentication,
Authorization,
Tokens, Signatures
(e.g., WS-Security)
Transaction
Transaction ID,
Atomicity,
Coordination
(e.g., WS-AtomicTransaction)
Routing
Target endpoint,
Action,
Message ID
(e.g., SOAPAction, To)
Reliability
Message ID,
Sequence,
Acknowledgment
(e.g., WS-ReliableMessaging)
Context
Application context,
Preferences,
Localization
(Custom headers)
Custom
Domain-specific
or application-
specific data
(Custom headers)
<soap:Header xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:wsse="http://docs.oasis-open.org/wss/...">
<wsse:Security> ... </wsse:Security> <!-- Security information -->
<wsa:MessageID>uuid:123e4567-e89b...</wsa:MessageID> <!-- Unique message identifier -->
<wsa:To>http://service.example.com/endpoint</wsa:To> <!-- Target endpoint -->
<wsrm:Sequence> ... </wsrm:Sequence> <!-- Reliability sequence -->
<app:Context> ... </app:Context> <!-- Application context -->
<app:CustomHeader> ... </app:CustomHeader> <!-- Custom header -->
</soap:Header>
Header Elements

SOAP Body

<soap:Body> (Required)

Carries the actual message payload

Request Message
(From Client to Server)

Payload
Contains the operation to be performed and input data

- Operation (Method)
- Input Parameters
- Request Data
- Validation Info

Response Message
(From Server to Client)

Payload
Contains the operation result and output data

- Result Status
- Response Data
- Fault (If Any)
- Additional Info

```xml
<soap:Body>
  <ns:GetAccountRequest>
    <ns:AccountId>12345</ns:AccountId>
    <ns:IncludeDetails>true</ns:IncludeDetails>
  </ns:GetAccountRequest>
  <ns:GetAccountResponse>
    <ns:Status>SUCCESS</ns:Status>
    <ns:Account>
      <ns:AccountId>12345</ns:AccountId>
      <ns:Balance>2500.75</ns:Balance>
    </ns:Account>
  </ns:GetAccountResponse>
</soap:Body>
```

Request Payload
Operation + Input Data

Response Payload
Result + Output Data

WSDL OVERVIEW

WSDL is an XML-based language that describes what a SOAP service does, how to access it, and what messages it exchanges.

WHAT IS WSDL?

- WSDL is an XML document that describes a web service.
- It acts as a contract between provider and consumer.
- It does not contain the implementation, only the interface.
- Used by tools to generate client proxies and server skeletons.

WSDL FILE STRUCTURE

```
<definitions>
  <types>...</types>
  <message>...</message>
  <portType>...</portType>
  <binding>...</binding>
  <service>...</service>
</definitions>
```

WSDL VERSIONS & VERSIONING STRATEGY

- WSDL 1.1 remains the most common version in enterprise SOAP tooling. WSDL 2.0 exists but has much lower adoption and tool support.
- Prefer document/literal wrapped contracts for broad interoperability unless an existing integration requires another style.
- Other service description languages: OpenAPI/Swagger and RAML describe REST APIs; Protocol Buffers describe gRPC services.
- Versioning strategy: bump the target namespace (e.g., .../customer/v2) or endpoint path for breaking changes; add new fields as optional to stay backward-compatible; keep prior versions live until consumers migrate.

KEY TAKEAWAY WSDL is the blueprint of a SOAP web service — it enables interoperability, automation, and clear communication.

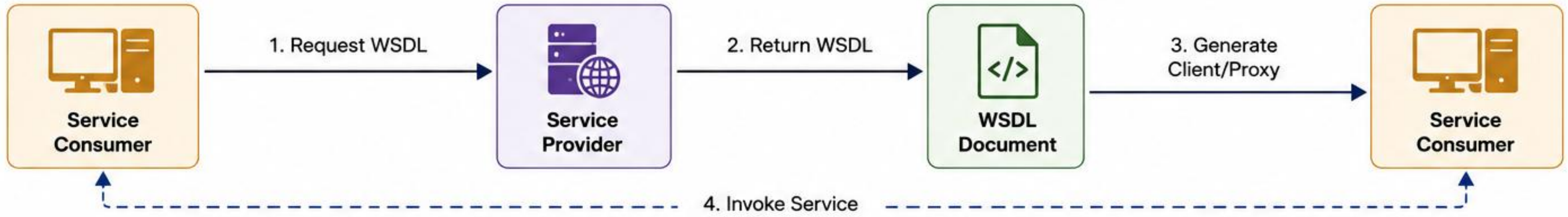
WSDL Overview

What is WSDL?

WSDL (Web Services Description Language) is an XML-based contract that describes a web service's functionality, endpoints, and data types.

- Contract-First Development
- Technology Independent
- Platform Independent
- Standardized Integration

How WSDL Works



WSDL Structure

<definitions>

- **<types>** Defines the data types (XSD Schemas)
- **<message>** Defines the input and output messages
- **<portType>** Defines the operations and message exchange
- **<binding>** Specifies the protocol and data format
- **<service>** Defines the service endpoint (URL)

WSDL Components

Types	Defines the XML Schema data types used.
Messages	Defines the input and output message parts.
PortType	Defines the operations (abstract interface).
Binding	Binds portType to a concrete protocol and data format.
Service	Defines the endpoint address where the service is available.

WSDL Example (Snippet)

```xml
<definitions name="OrderService"
  targetNamespace="http://example.com/order"
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  xmlns:tns="http://example.com/order"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <types>...</types>
  <message name="GetOrderRequest">...</message>
  <message name="GetOrderResponse">...</message>
  <portType name="OrderPortType">...</portType>
  <binding name="OrderBinding" type="tns:OrderPortType">
    ...
  </binding>
  <service name="OrderService">
    <port name="OrderPort" binding="tns:OrderBinding">
      <soap:address location="http://example.com/order"/>
    </port>
  </service>
</definitions>
```

WSDL COMPONENTS

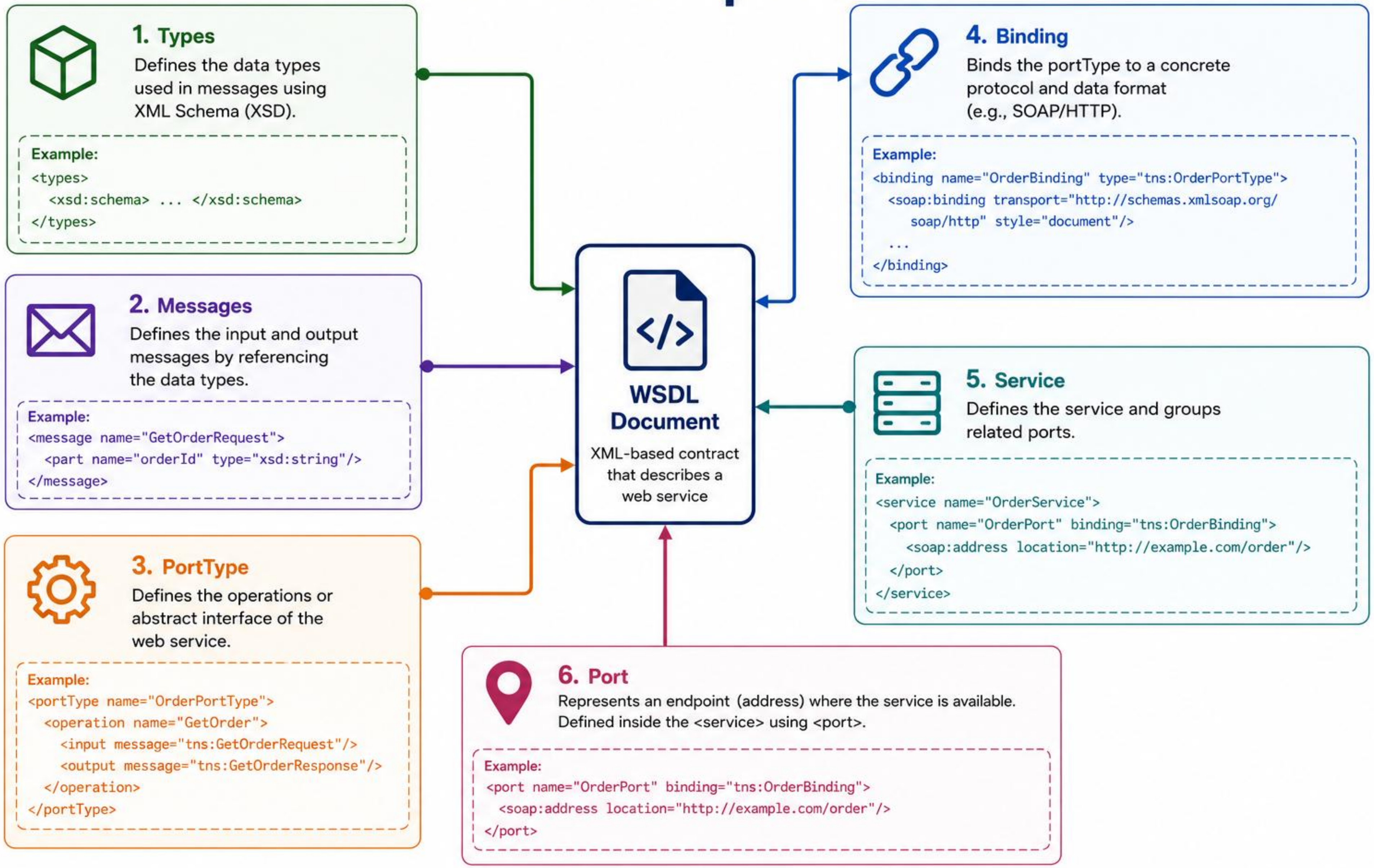
Each WSDL component has a specific role in describing how the service works and how clients interact with it.

COMPONENT	PURPOSE	DEPENDS ON
`definitions`	Root element that wraps the entire WSDL document	None (root)
`types`	Defines data types used in messages (XSD)	definitions
`message`	Defines the data being sent or received	types
`portType`	Defines a set of operations supported	message
`binding`	Binds a portType to a protocol (SOAP) and format	portType
`service`	Defines the endpoint address (URL)	binding

ABSTRACT LAYER (TYPES, MESSAGE, PORTTYPE) DEFINES WHAT THE SERVICE DOES; CONCRETE LAYER (BINDING, SERVICE) DEFINES HOW AND WHERE IT'S ACCESSED.

KEY TAKEAWAY WSDL is a contract-first approach — each component has a specific responsibility, together describing the full service.

WSDL Components



XSD SCHEMA DESIGN

XML Schema (XSD) defines the structure, data types, constraints, and relationships of XML documents.

XSD DESIGN PRINCIPLES

- Reusable complexType and simpleType definitions for shared structures.
- xs:sequence controls child-element order within a complexType.
- Constrain values: enumerations, pattern restrictions, length limits.
- Use elementFormDefault="qualified"; design namespaces deliberately.
- Choose correct data types; use meaningful, consistent names.

OCCURRENCE INDICATORS

VALUE	MEANING
`minOccurs="0"`	Element is optional
`maxOccurs="1" (default)`	Element appears once
`maxOccurs="unbounded"`	Element can repeat
`nillable="true"`	PRESENT with xsi:nil="true" — different from minOccurs="0" (ABSENT)

WHY DESIGN A GOOD XSD SCHEMA?

Data Validation

Ensures XML conforms to rules.

Reusability

Schemas reused across apps.

Interoperability

Common contract for exchange.

KEY TAKEAWAY Well-designed XSD schemas ensure data quality, consistency, and interoperability — validate early and often.

XSD Schema Design

1. Requirements
- Identify data
- Define rules
- Determine relationships

→

2. Conceptual Model
- Entities
- Attributes
- Relationships

→

3. XSD Schema
- Elements
- Types
- Constraints

→

4. XML Instance
- Valid XML
- Follows schema
- Contains data

→

5. Validation
- Validate XML
- Enforce rules
- Ensure integrity

XSD Schema Structure

<xs:schema>

- **<xs:element>** Defines root and child elements
- **<xs:complexType>** Defines complex data types
- **<xs:simpleType>** Defines simple data types
- **<xs:attribute>** Defines attributes
- **<xs:restriction> / <xs:enumeration>** Defines constraints and enumerations

</xs:schema>

XSD Building Blocks

Component	Purpose
Element	Defines data structures and hierarchy
ComplexType	Groups elements and/or attributes
SimpleType	Defines primitive data with constraints
Attribute	Provides additional information
Restriction	Restricts values based on rules (length, pattern, etc.)
Enumeration	Defines a list of allowed values

XSD Schema Example

```xml
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="Customer">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="Id" type="xs:int"/>
        <xs:element name="Name" type="xs:string"/>
        <xs:element name="Email" type="xs:string"/>
      </xs:sequence>
      <xs:attribute name="status">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:enumeration value="Active"/>
            <xs:enumeration value="Inactive"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:attribute>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Common Data Types

xs:string | xs:int | xs:decimal | xs:date | xs:boolean | xs:anyType

Common Constraints

minLength | maxLength | pattern | minInclusive | maxInclusive | enumeration

CONTRACT-FIRST DESIGN

Contract-first and code-first are both valid approaches — choose based on the integration's needs.

CONTRACT-FIRST

- Best for shared external contracts
- Strong control over XML and namespaces
- Better consumer/provider coordination
- Requires schema-design skills
- Good for long-lived integrations

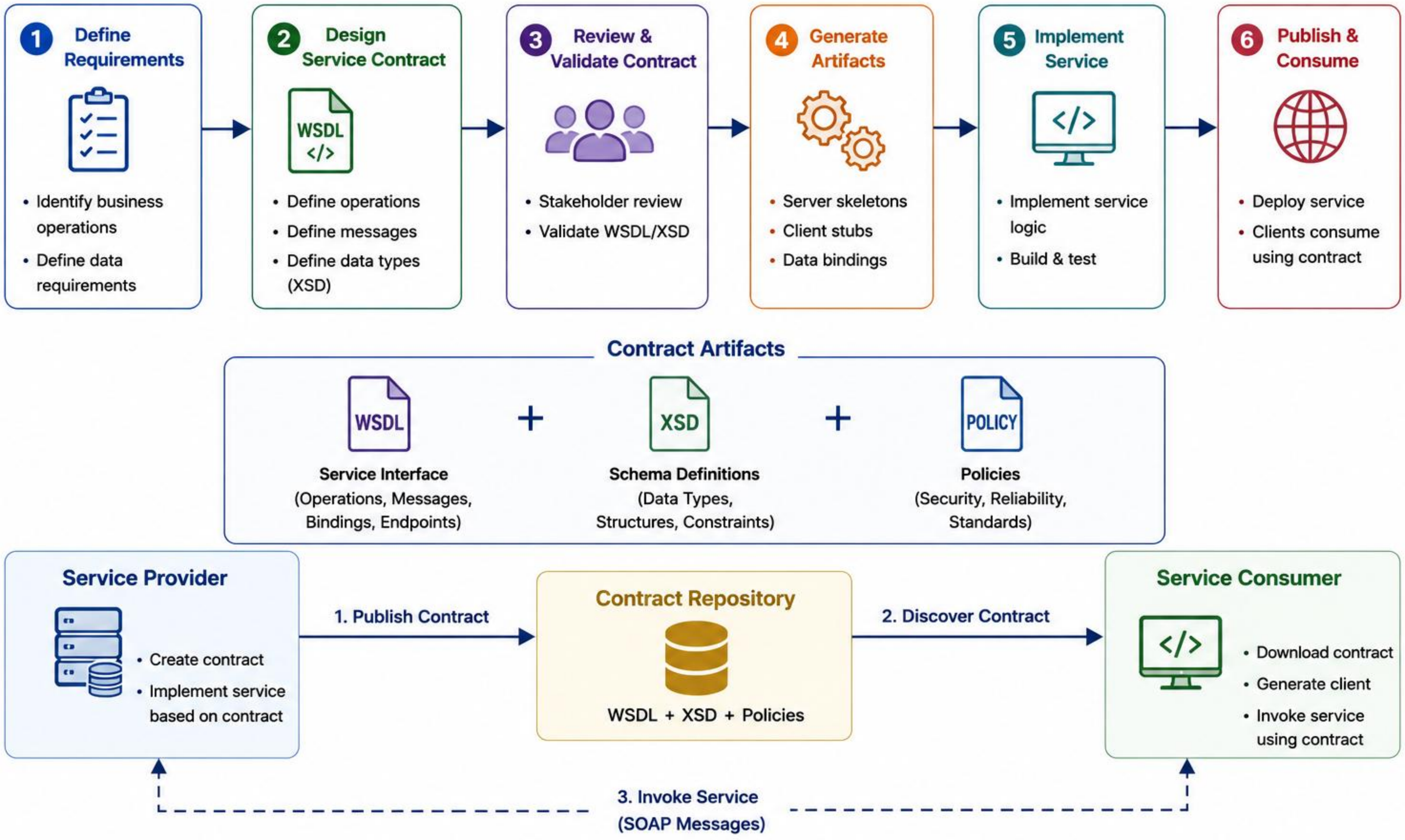
VS

CODE-FIRST

- Useful for internal/simple services
- Faster initial implementation
- Contract generated from code
- May expose implementation details
- Good for prototypes or controlled consumers

KEY TAKEAWAY Contract-first suits shared, long-lived integrations; code-first suits internal or fast-moving services — pick based on your consumers and constraints.

Contract-First Design



SOAP REQUEST–RESPONSE LIFECYCLE

A SOAP message travels from client to service and back through seven well-defined steps.

1. **Marshal to XML** — Client marshals the request object to XML.
2. **Create Envelope** — SOAP Envelope is created.
3. **Apply Headers** — Headers and security are applied.
4. **Send** — Message is sent over transport.
5. **Validate & Process** — Service validates and processes it.
6. **Generate Result** — Response or Fault is generated.
7. **Unmarshal** — Client unmarshals the result.

BEST PRACTICES

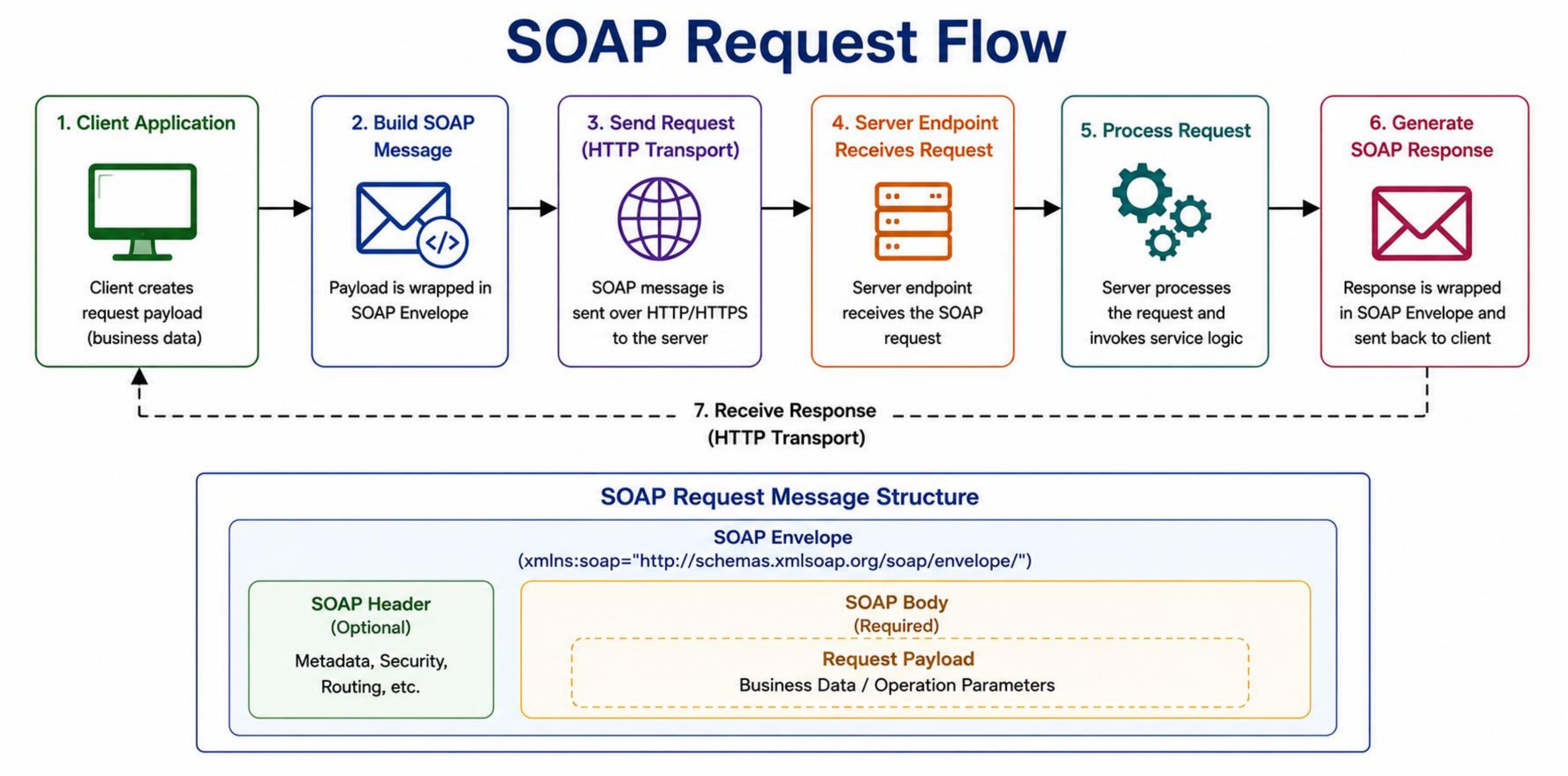
- Always use HTTPS for secure communication.
- Validate requests at the service boundary.
- Keep the SOAP request size reasonable; handle timeouts gracefully.

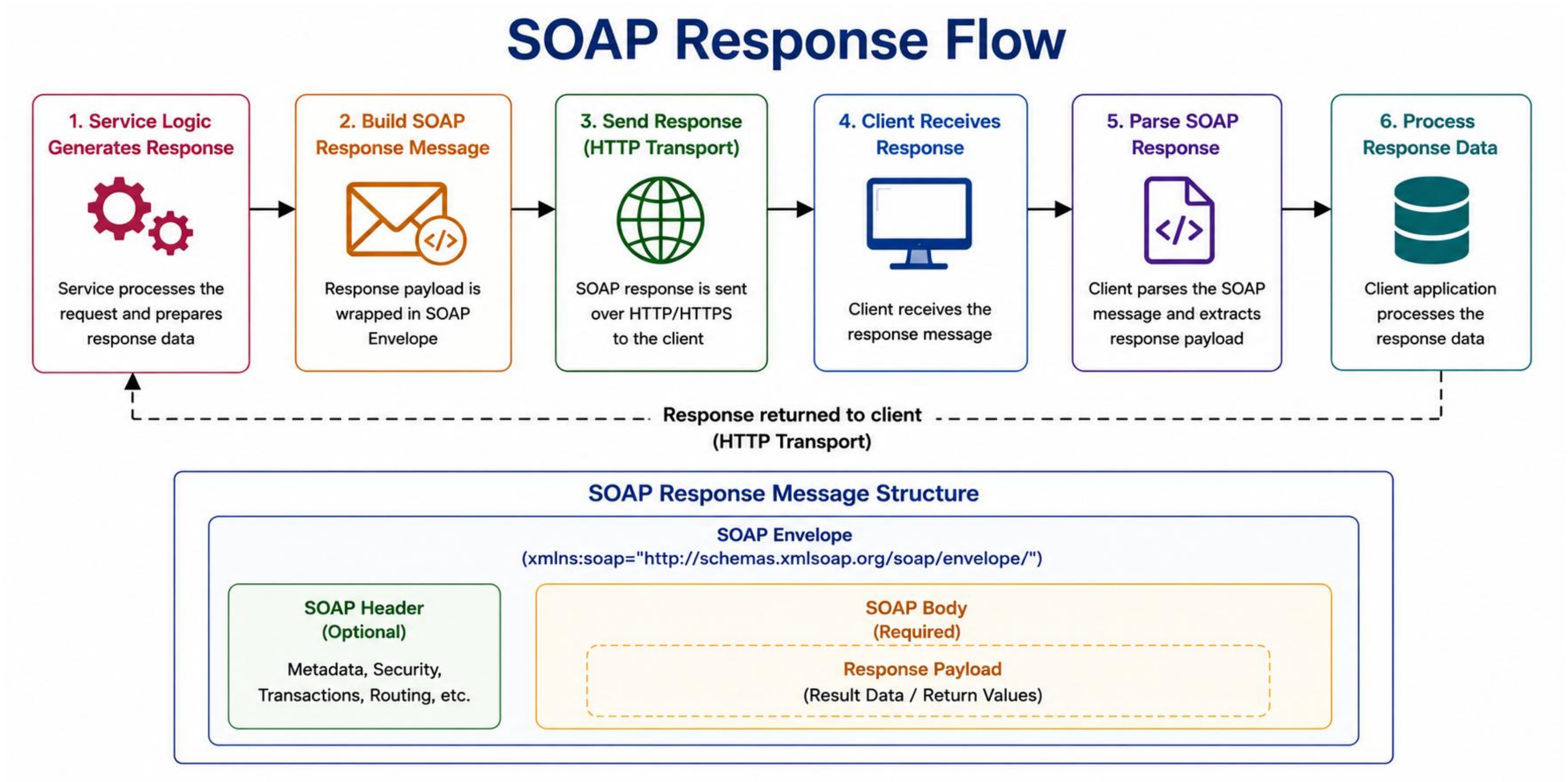
TYPICAL HTTP OUTCOMES

Situation	Typical HTTP outcome
Successful SOAP response	200 OK
Authentication rejected at HTTP layer	401 or 403
Malformed request / SOAP 1.2 sender fault	Often 400
SOAP processing/server fault	Often 500

The SOAP Fault payload is the primary service-level error contract; HTTP status usage varies by version and framework.

KEY TAKEAWAY The SOAP request-response lifecycle moves through marshaling, enveloping, transport, processing, and unmarshaling — with the SOAP Fault carrying service-level errors.


SOAP Request Flow
1. Client Application
Client creates request payload (business data)
2. Build SOAP Message
Payload is wrapped in SOAP Envelope
3. Send Request (HTTP Transport)
SOAP message is sent over HTTP/HTTPS to the server
4. Server Endpoint Receives Request
Server endpoint receives the SOAP request
5. Process Request
Server processes the request and invokes service logic
6. Generate SOAP Response
Response is wrapped in SOAP Envelope and sent back to client
7. Receive Response (HTTP Transport)
SOAP Request Message Structure
SOAP Envelope
(xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/")
SOAP Header (Optional)
Metadata, Security, Routing, etc.
SOAP Body (Required)
Request Payload
Business Data / Operation Parameters


SOAP Response Flow
1. Service Logic Generates Response
Service processes the request and prepares response data
2. Build SOAP Response Message
Response payload is wrapped in SOAP Envelope
3. Send Response (HTTP Transport)
SOAP response is sent over HTTP/HTTPS to the client
4. Client Receives Response
Client receives the response message
5. Parse SOAP Response
Client parses the SOAP message and extracts response payload
6. Process Response Data
Client application processes the response data
Response returned to client (HTTP Transport)
SOAP Response Message Structure
SOAP Envelope
(xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/")
SOAP Header (Optional)
Metadata, Security, Transactions, Routing, etc.
SOAP Body (Required)
Response Payload
(Result Data / Return Values)

ENTERPRISE SOAP USE CASES

SOAP Web Services continue to power mission-critical enterprise integrations where reliability and security are non-negotiable.

Financial Services

Core banking host integration, payment gateway transactions, and settlement systems.

Healthcare

Healthcare partner contracts for patient, lab, and billing data exchange (HIPAA).

Insurance

Insurance policy and claims exchange across agents, carriers, and adjudication systems.

Retail & E-commerce

ERP and supply-chain integrations for order, inventory, and fulfillment data.

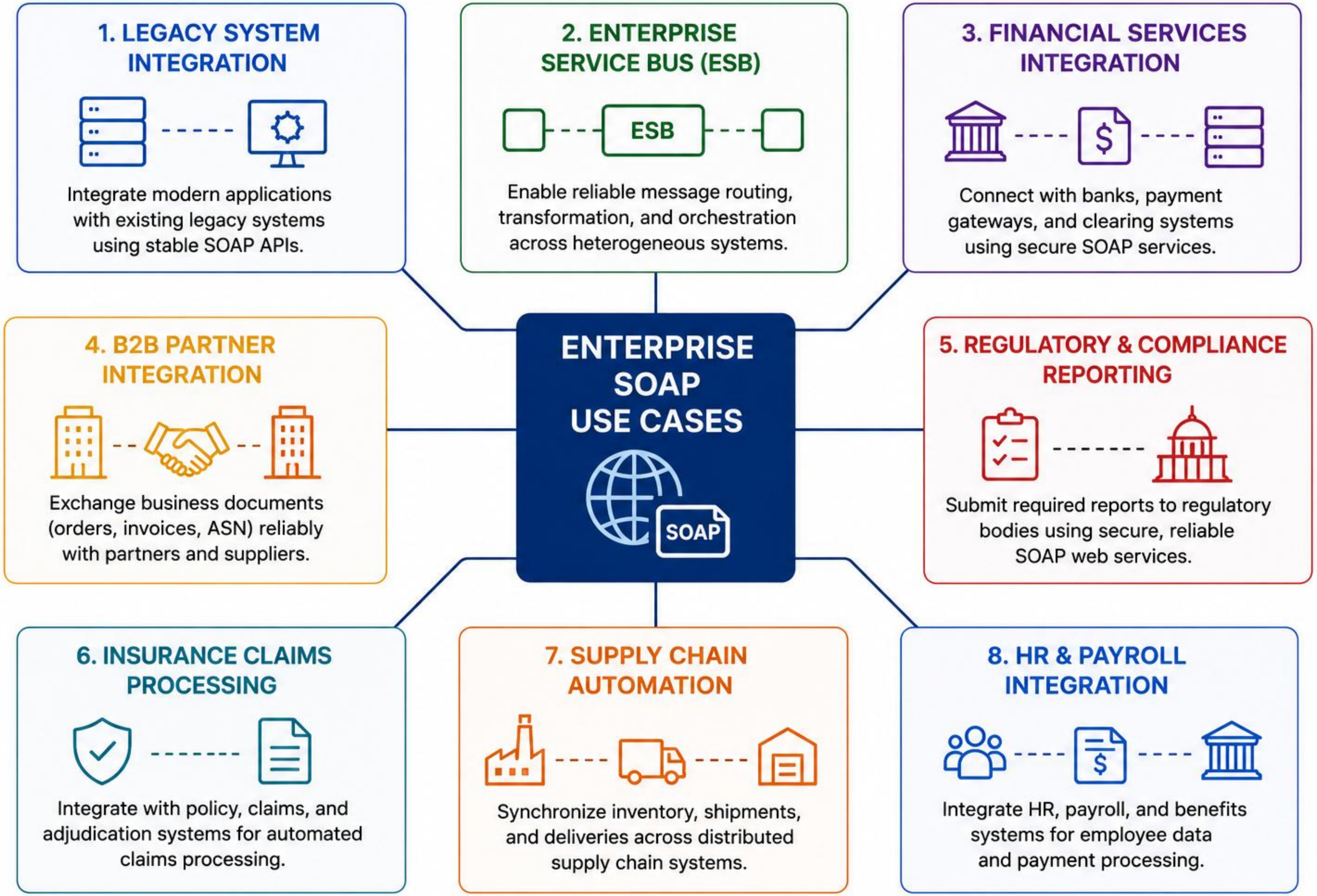
Government

Government B2B interfaces for secure cross-department and citizen-service data exchange.

Manufacturing

Legacy ESB integration linking plant systems with supply-chain and partner networks.

KEY TAKEAWAY SOAP remains a trusted choice for enterprise integrations demanding security, reliability, and standards.


1. LEGACY SYSTEM INTEGRATION
Integrate modern applications with existing legacy systems using stable SOAP APIs.
2. ENTERPRISE SERVICE BUS (ESB)
ESB
Enable reliable message routing, transformation, and orchestration across heterogeneous systems.
3. FINANCIAL SERVICES INTEGRATION
$
Connect with banks, payment gateways, and clearing systems using secure SOAP services.
ENTERPRISE SOAP USE CASES
SOAP
4. B2B PARTNER INTEGRATION
Exchange business documents (orders, invoices, ASN) reliably with partners and suppliers.
5. REGULATORY & COMPLIANCE REPORTING
Submit required reports to regulatory bodies using secure, reliable SOAP web services.
6. INSURANCE CLAIMS PROCESSING
Integrate with policy, claims, and adjudication systems for automated claims processing.
7. SUPPLY CHAIN AUTOMATION
Synchronize inventory, shipments, and deliveries across distributed supply chain systems.
8. HR & PAYROLL INTEGRATION
$
Integrate HR, payroll, and benefits systems for employee data and payment processing.

SECURITY CONSIDERATIONS

SOAP services exchange critical business data — security must be built in at every layer: transport, message, and service.

Transport Layer

- Use HTTPS with currently approved TLS versions and cipher suites per organizational policy.

Message Layer (WS-Security)

- UsernameToken, X.509 certificates, XML Signature; guard against XML Signature Wrapping and replay (timestamps, nonces).

Confidentiality

- XML Encryption for sensitive elements.

Service Layer

- Role-based access control, input validation, and schema validation before processing.

XML Parser Hardening

- Disable external entities (XXE); limit entity expansion, payload size, and nesting depth.

Logging & Monitoring

- Log metadata, correlation IDs, operation names, status, and safe diagnostic context. Redact or avoid sensitive message content.

WS-SECURITY: WHAT PROTECTS WHAT

Authentication tokens (e.g., UsernameToken) — identify the caller.

XML Signature — protects message integrity and authenticity.

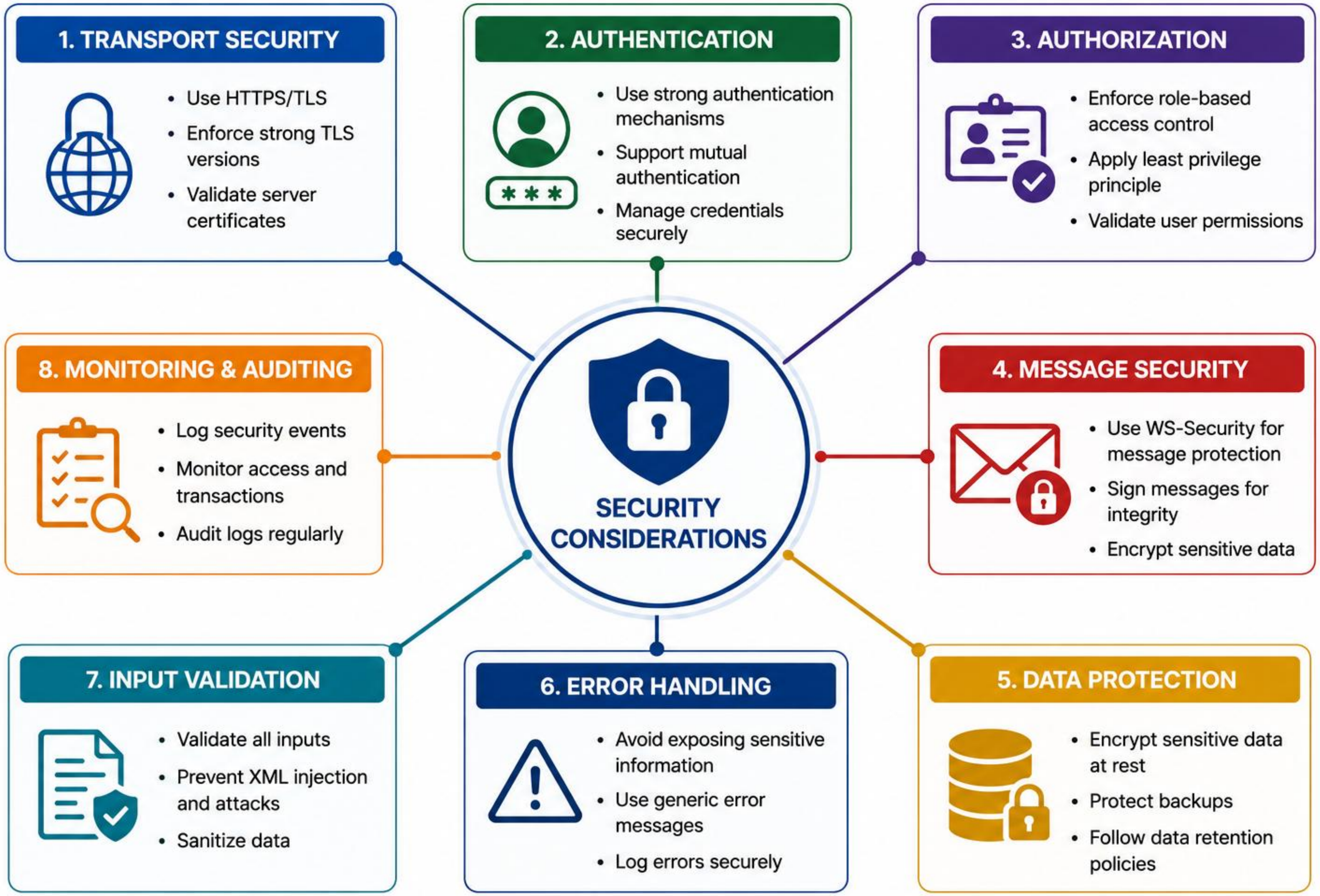
XML Encryption — protects message-level confidentiality.

Timestamps / unique message IDs — prevent replay.

HTTPS/TLS — protects the transport.

WS-Security — can protect the message end-to-end, independent of transport.

KEY TAKEAWAY Security in SOAP is multi-layered — secure the transport, protect the message, harden the service, monitor continuously.


SECURITY CONSIDERATIONS
1. TRANSPORT SECURITY
Use HTTPS/TLS
Enforce strong TLS versions
Validate server certificates
2. AUTHENTICATION
Use strong authentication mechanisms
Support mutual authentication
Manage credentials securely
3. AUTHORIZATION
Enforce role-based access control
Apply least privilege principle
Validate user permissions
4. MESSAGE SECURITY
Use WS-Security for message protection
Sign messages for integrity
Encrypt sensitive data
5. DATA PROTECTION
Encrypt sensitive data at rest
Protect backups
Follow data retention policies
6. ERROR HANDLING
Avoid exposing sensitive information
Use generic error messages
Log errors securely
7. INPUT VALIDATION
Validate all inputs
Prevent XML injection and attacks
Sanitize data
8. MONITORING & AUDITING
Log security events
Monitor access and transactions
Audit logs regularly

LAB OVERVIEW – SOAP API DESIGN

Design a contract-first SOAP interface (WSDL + XSD) for Northstar's CRM customer service — no server required.

BUSINESS SCENARIO

- Northstar's CRM must publish a stable SOAP contract before a billing partner integrates — architects freeze the XML and operations before Lab 24 builds the endpoint.
- Operations: CreateCustomer, UpdateCustomer, GetCustomer.

SCOPE & CONSTRAINTS

- Contract artifacts: customer.xsd + CustomerService.wsdl.
- No live server required — design-time validation only.
- Feeds Lab 24, which implements the endpoint with Spring-WS.

TOOLS & TECHNOLOGIES

XML/WSDL Editor
Primary contract editor

xmllint (Validation)
or IDE schema validation

SoapUI
WSDL-driven testing (primary)

Postman / curl
Manual transport testing only

Maven is only needed if the project uses schema-validation plugins or generated-code tooling (e.g., wsimport, JAXB).

KEY TAKEAWAY This lab follows contract-first design: WSDL defines the contract, XSD defines the data — Lab 24 implements it with Spring-WS.

LAB TASKS AND DELIVERABLES

Follow a contract-first approach to design a SOAP contract for Northstar's CRM customer service.

#	TASK	DETAILS	FORMAT
1	Project Setup	Create lab13-crm layout; write operation-matrix.md.	Document
2	Design XSD Schema	customer.xsd: CustomerType, CustomerStatus, request/response elements.	.xsd
3	Design WSDL Contract	CustomerService.wsdl: messages, portType, binding, service.	.wsdl
4	Write Sample Envelopes	Create/Update/Get requests & responses for CUS-1001, CUS-1002.	.xml
5	Write Fault Samples	Not-found and validation fault envelopes.	.xml
6	Document & Handoff	soap-design-notes.md + partner handoff checklist.	Document

EVALUATION CRITERIA — CONTRACT QUALITY

- Stable, versioned targetNamespace.
- elementFormDefault="qualified".
- No duplicate global element names.
- Reusable shared types (e.g., CustomerType).
- Document/literal wrapped binding.
- One request and one response element per operation.
- Typed SOAP Fault detail elements.
- Backward-compatible schema choices.

EVALUATION CRITERIA — FAULTS & VALIDATION

Business faults (not-found, invalid status, duplicate, validation) — defined in the WSDL.
Technical faults (unavailable, DB failure, server error) — generic fault policy is OK.
Valid endpoint address placeholder in <service>.
Validates successfully without network access.
CreateCustomer/UpdateCustomer/GetCustomer correctly defined.
Sample and fault envelopes are complete.

KEY TAKEAWAY This lab helps you design and document a partner-ready SOAP contract — Lab 24 implements it with Spring-WS.

MODULE SUMMARY — DEFINITION OF DONE

A SOAP contract is ready when it meets this Definition of Done -- contract-first, end to end.

1. Target namespace is stable and versioned.
2. XSD types reflect business meaning.
3. Optionality and cardinality are intentional.
4. Operations have explicit request and response elements.
5. Business faults are contractually defined.
6. WSDL uses interoperable binding conventions.
7. Sample envelopes validate against the contract.
8. Security requirements are documented.
9. Sensitive information is not logged.
10. Backward compatibility is reviewed.
11. Partner handoff documentation is complete.

KEY TAKEAWAY Use this checklist before publishing any SOAP contract — it turns module concepts into a concrete, reviewable definition of done.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of SOAP web services, WSDL, and contract-first design.

1. What's the primary purpose of SOAP?

A. Lightweight messaging over HTTP

B. Structured messaging using XML

C. Real-time event streaming

D. Defining RESTful APIs

2. Which WSDL component defines message data types?

A. <service>

B. <binding>

C. <types>

D. <portType>

3. What does XSD stand for?

A. XML Service Definition

B. XML Schema Definition

C. XML Structure Document

D. XML Style Descriptor

4. In contract-first approach, what's created first?

A. Client code

B. Database schema

C. Service implementation

D. Service contract (WSDL and XSD)

KEY TAKEAWAY SOAP is a structured XML messaging protocol — contract-first means the WSDL and XSD come before any code.

KNOWLEDGE CHECK (2 OF 2)

Four more questions, plus the full answer key.

5. Which namespace-qualified element is the root of a SOAP message?

A. SOAP Envelope (namespace-qualified)

B. <Message>

C. <Request>

D. <SOAPBody>

6. What does the SOAP Body contain?

A. HTTP headers

B. Authentication tokens

C. The actual request or response message

D. Routing information

7. How does a SOAP service indicate an error?

A. By returning HTTP 404

B. By throwing a Java exception only

C. By returning a SOAP Fault message

D. By closing the connection

8. Where is a SOAP processing error represented in the message?

A. As a SOAP Fault inside the SOAP Body

B. In the HTTP status code alone

C. In a custom HTTP header

D. It cannot be represented in the message

ANSWER KEY

- **1-B 2-C 3-B 4-D 5-A 6-C 7-C 8-A**

KEY TAKEAWAY SOAP delivers where security, reliability, compliance, and long-term stability matter most.

MODULE 13 COMPLETE

SOAP API Design with Java

You can now design, implement, secure, and consume enterprise-grade SOAP web services with confidence.